Data Mining Lab 2
Name:Sameer Gupta
Roll No:252010036

February 17, 2026

[1]:
```
#IRIS DATASET
#1. Load the dataset and display the first 10 records.
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
# Load dataset
iris = load_iris()
# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target
# Display first 10 records
df.head(10)
```

[1]:
```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2
5                5.4               3.9                1.7               0.4
6                4.6               3.4                1.4               0.3
7                5.0               3.4                1.5               0.2
8                4.4               2.9                1.4               0.2
9                4.9               3.1                1.5               0.1

   target
0       0
1       0
2       0
3       0
4       0
5       0
6       0
7       0
8       0
9       0
```

[2]:
```python
#2. What are the feature names
iris.feature_names
```

[2]:
```
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

[3]:
```python
#3. What are the target classes?
iris.target_names
```

[3]:
```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

[4]:
```python
#4. Compute summary statistics (mean, median, std, min,max) for each feature.
summary = df.describe()
summary.loc['median'] = df.median()
summary
```

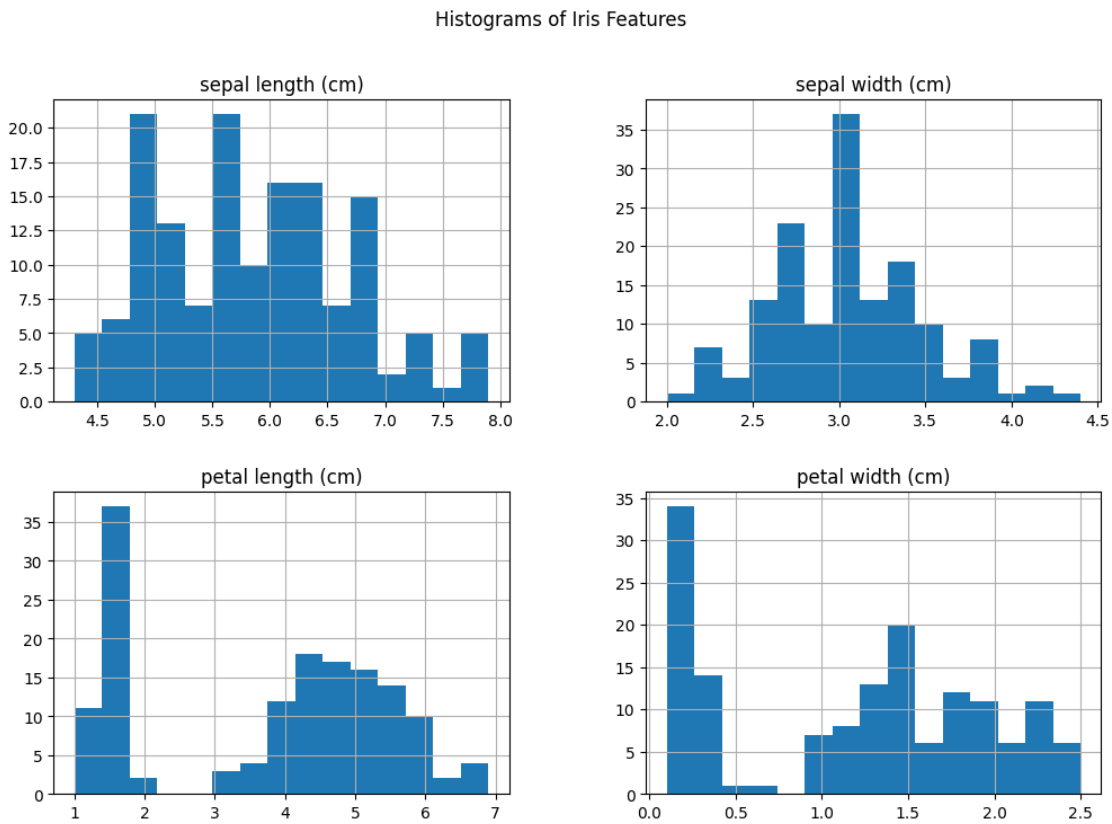
[4]:
```
        sepal length (cm)  sepal width (cm)  petal length (cm)  \
count          150.000000        150.000000         150.000000
mean             5.843333          3.057333           3.758000
std              0.828066          0.435866           1.765298
min              4.300000          2.000000           1.000000
25%              5.100000          2.800000           1.600000
50%              5.800000          3.000000           4.350000
75%              6.400000          3.300000           5.100000
max              7.900000          4.400000           6.900000
median           5.800000          3.000000           4.350000

        petal width (cm)      target
count         150.000000  150.000000
mean            1.199333    1.000000
std             0.762238    0.819232
min             0.100000    0.000000
25%             0.300000    0.000000
50%             1.300000    1.000000
75%             1.800000    2.000000
max             2.500000    2.000000
median          1.300000    1.000000
```

[5]:
```python
#5.Check for missing values. Are there any?
df.isnull().sum()
# Result: No missing values in the Iris dataset.
```

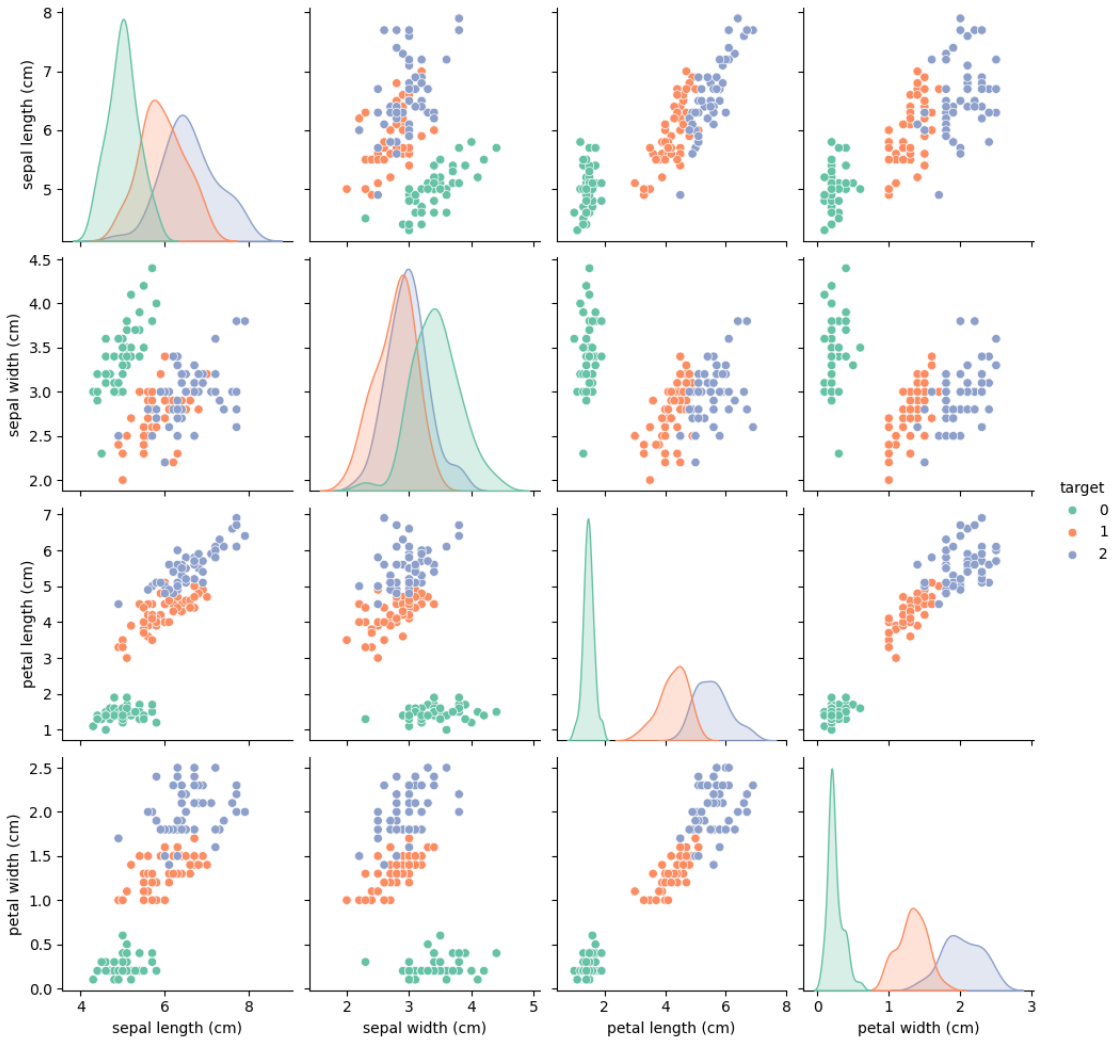
[5]:
```
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
```

```
target                0
dtype: int64
```

[6]:
```
#6. Plot histograms for all features. What patterns do you observe?
df.iloc[:, :-1].hist(bins=15, figsize=(12, 8))
plt.suptitle("Histograms of Iris Features")
plt.show()
#Observation (brief):
    #1.Petal features show clear class separation
    #2.Sepal features overlap more
```



[7]:
```
#7. Create a pair plot.
sns.pairplot(df, hue='target', palette='Set2')
plt.show()
```

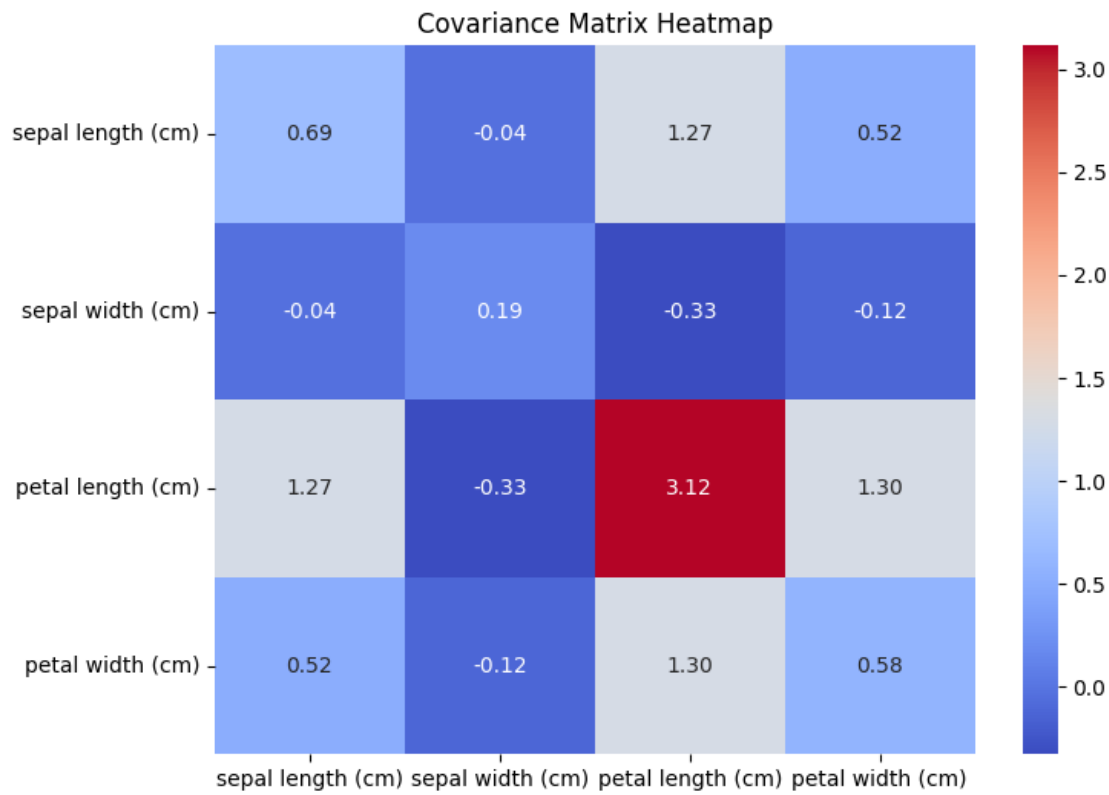


[8]:
```
#8. Create scatter plot.
plt.figure(figsize=(8,6))
sns.scatterplot(
    x='petal length (cm)',
    y='petal width (cm)',
    hue='target',
    data=df,
    palette='Set1'
)
plt.title("Petal Length vs Petal Width")
plt.show()
```

[9]:
```
#9. Which features separate classes best?
# Visual inspection via pairplot indicates:
best_features = ["petal length (cm)", "petal width (cm)"]
best_features
#Answer:Petal length and petal width separate classes best (especially Setosa).
```

[9]: ['petal length (cm)', 'petal width (cm)']

[10]:
```
#10. Give a heatmap for covariance matrix.
plt.figure(figsize=(8,6))
sns.heatmap(
    df.iloc[:, :-1].cov(),
    annot=True,
    cmap='coolwarm',
    fmt=".2f"
)
plt.title("Covariance Matrix Heatmap")
plt.show()
```



[11]:
```python
#WINE DATASET
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
# Load dataset
wine = load_wine()

# Create DataFrame
df = pd.DataFrame(wine.data, columns=wine.feature_names)
df['target'] = wine.target
df.head()
```

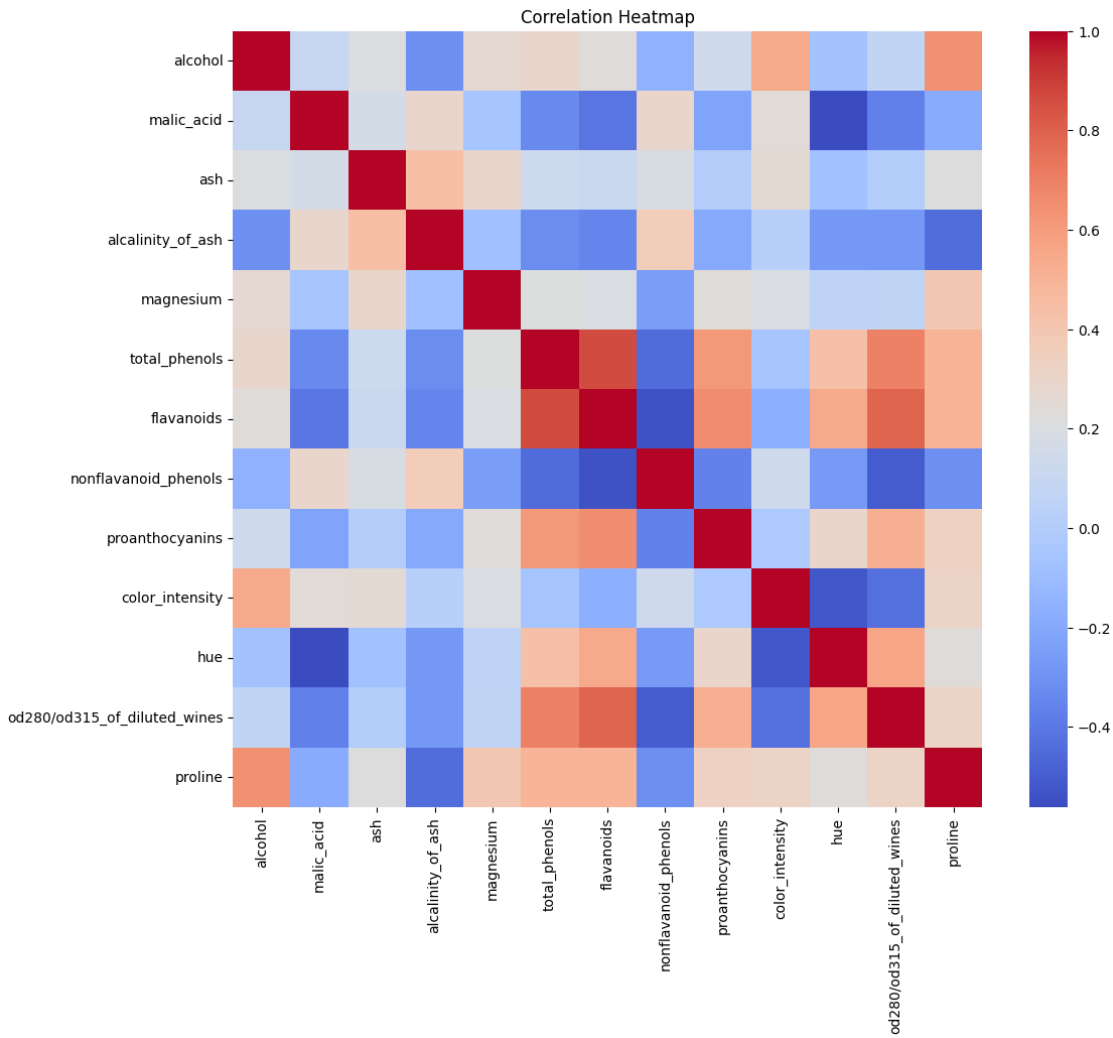
[11]:
```
   alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols  \
0    14.23        1.71  2.43               15.6      127.0           2.80
1    13.20        1.78  2.14               11.2      100.0           2.65
2    13.16        2.36  2.67               18.6      101.0           2.80
```

```
3    14.37        1.95  2.50                16.8      113.0           3.85
4    13.24        2.59  2.87                21.0      118.0           2.80

   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \
0        3.06                  0.28             2.29             5.64  1.04
1        2.76                  0.26             1.28             4.38  1.05
2        3.24                  0.30             2.81             5.68  1.03
3        3.49                  0.24             2.18             7.80  0.86
4        2.69                  0.39             1.82             4.32  1.04

   od280/od315_of_diluted_wines  proline  target
0                          3.92   1065.0       0
1                          3.40   1050.0       0
2                          3.17   1185.0       0
3                          3.45   1480.0       0
4                          2.93    735.0       0
```

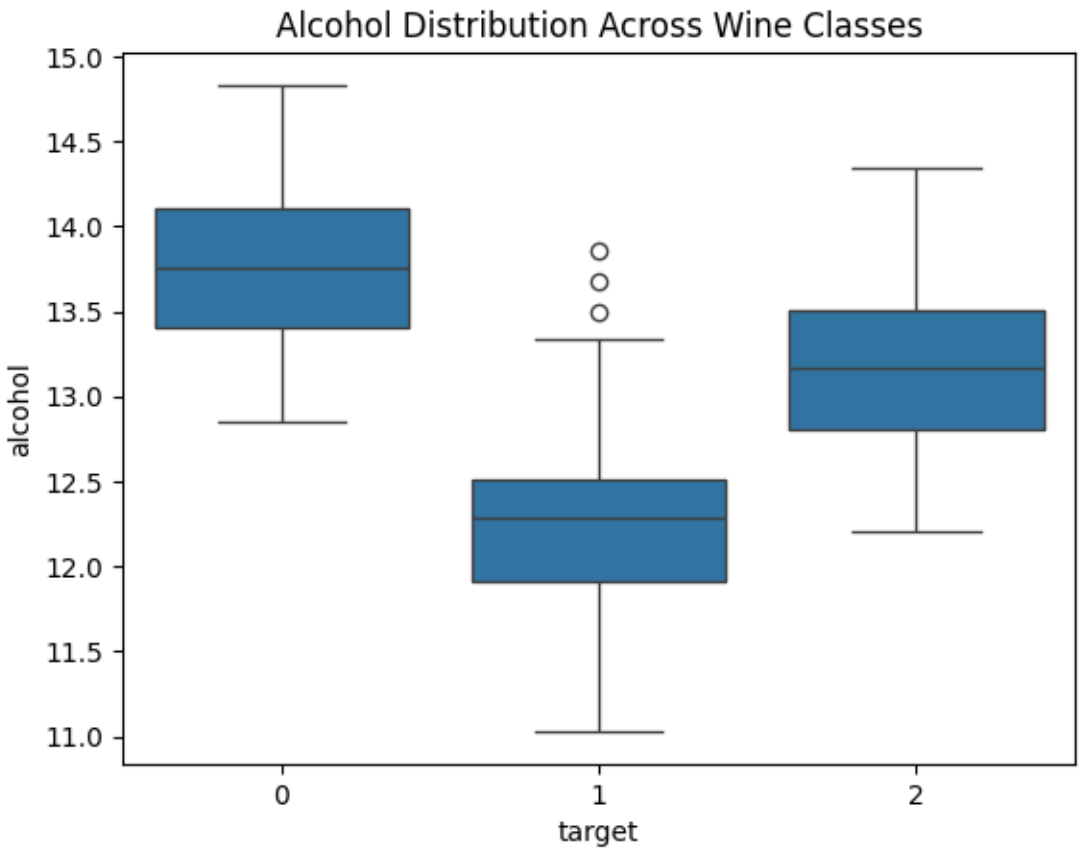
[12]:
```
#1.Identify the number of samples per target classes.
df['target'].value_counts().sort_index()
```

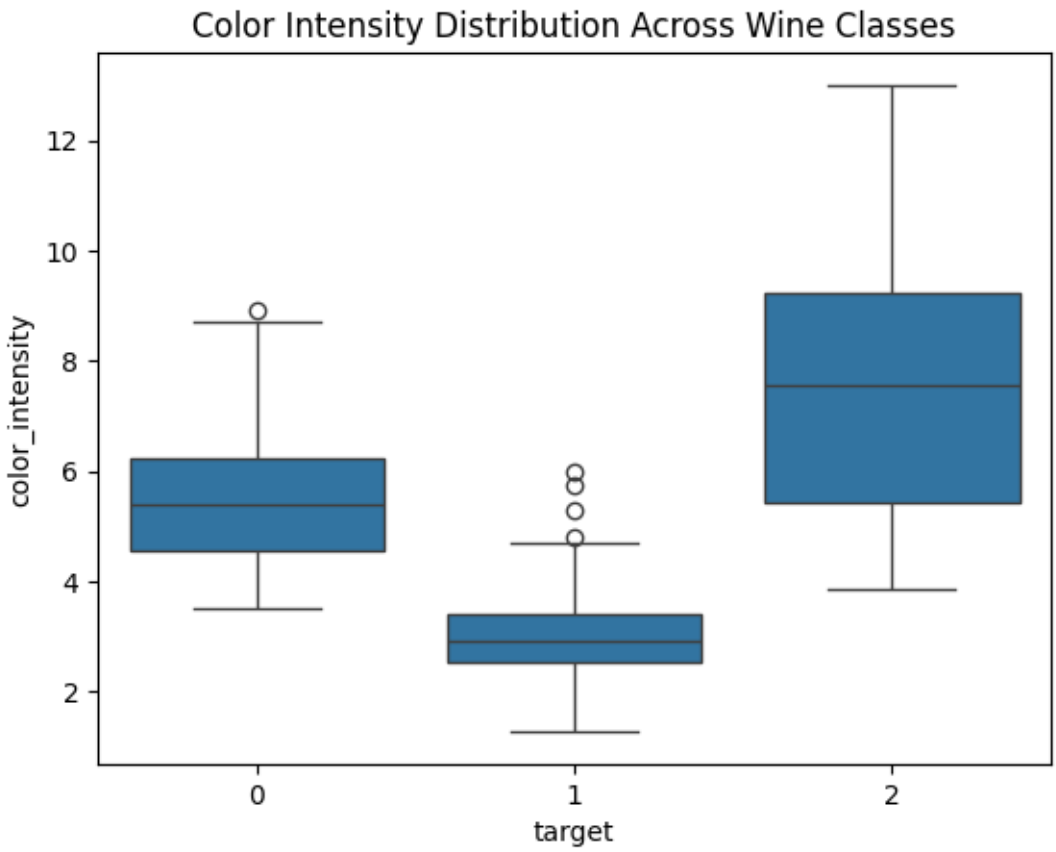
[12]:
```
target
0    59
1    71
2    48
Name: count, dtype: int64
```

[13]:
```
#2.Plot the correlation heatmap. Identify the highly correlated features?
plt.figure(figsize=(12,10))
sns.heatmap(df.iloc[:, :-1].corr(), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
#Observation:
    #1.High correlation between:
    #2.flavanoids & total_phenols
    #3.od280/od315_of_diluted_wines & flavanoids
    #4.color_intensity & hue (negative)
```



[14]:
```
#3.Compare feature distributions across different wine classes.
sns.boxplot(x='target', y='alcohol', data=df)
plt.title("Alcohol Distribution Across Wine Classes")
plt.show()

sns.boxplot(x='target', y='color_intensity', data=df)
plt.title("Color Intensity Distribution Across Wine Classes")
plt.show()
```

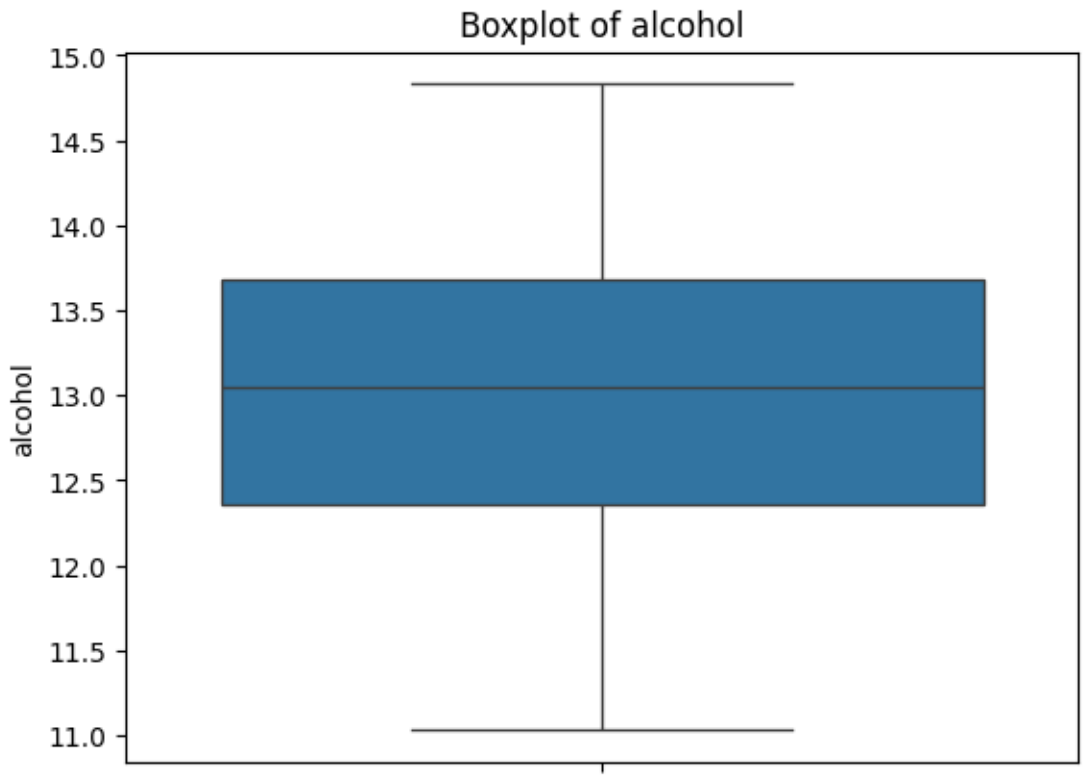

Alcohol Distribution Across Wine Classes
15.0
14.5
14.0
13.5
13.0
12.5
12.0
11.5
11.0
alcohol
0
1
2
target

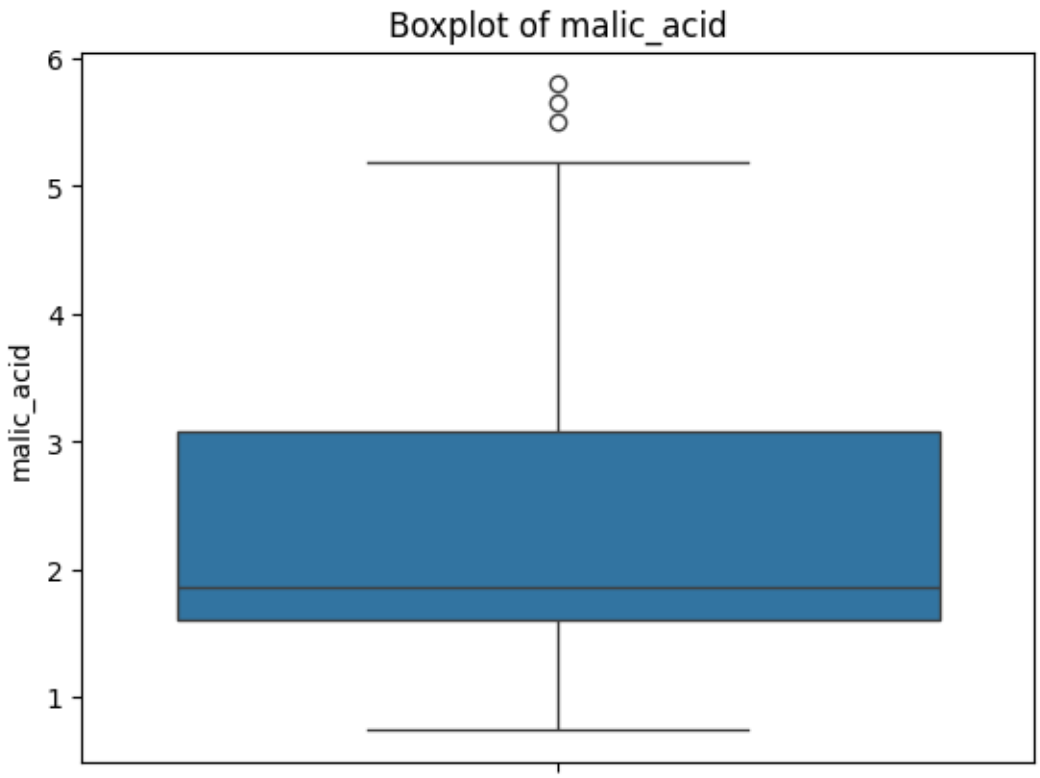


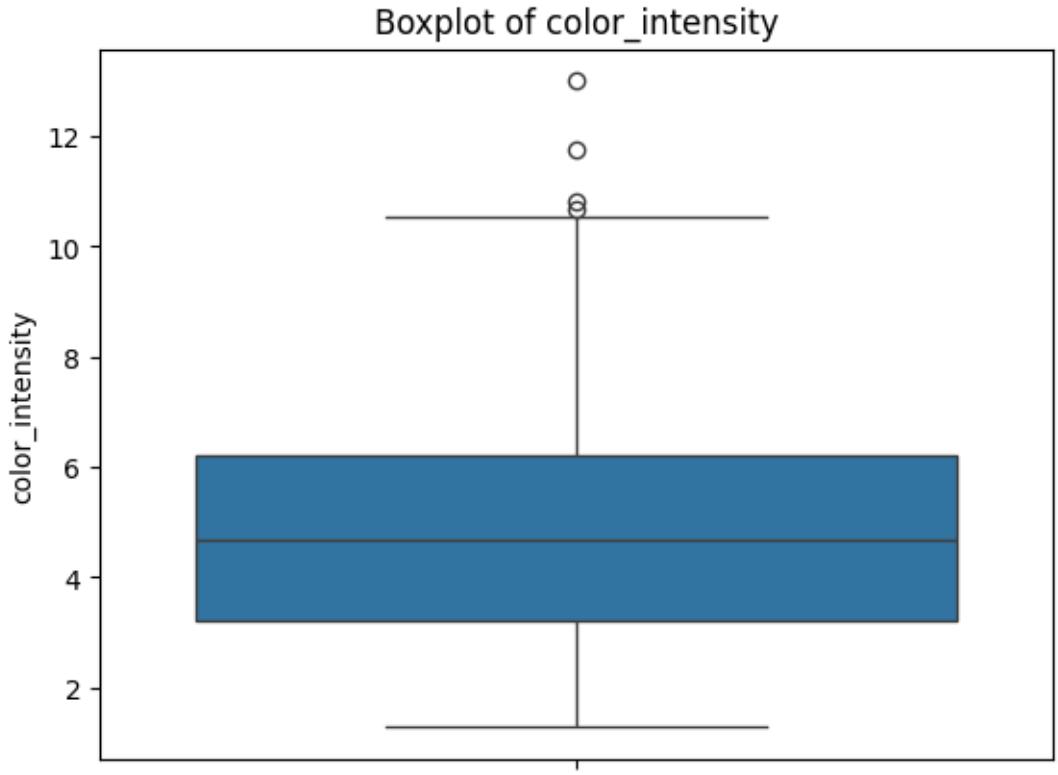
[15]:
```
#4.Identify features with highest variance. Why might they be important?
df.iloc[:, :-1].var().sort_values(ascending=False)
#Why important (brief):
    #1.High variance features (e.g. proline, color_intensity)
    #2.Carry more discriminatory information → useful for classification
```

[15]:
```
proline                       99166.717355
magnesium                       203.989335
alcalinity_of_ash                11.152686
color_intensity                   5.374449
malic_acid                        1.248015
flavanoids                        0.997719
alcohol                           0.659062
od280/od315_of_diluted_wines      0.504086
total_phenols                     0.391690
proanthocyanins                   0.327595
ash                               0.075265
hue                               0.052245
nonflavanoid_phenols              0.015489
dtype: float64
```

[16]:
```
#5.Use boxplots to detect outliers in at least three features.
features = ['alcohol', 'malic_acid', 'color_intensity']
for f in features:
    sns.boxplot(y=df[f])
    plt.title(f"Boxplot of {f}")
    plt.show()
```




Boxplot of malic_acid
malic_acid
6
5
4
3
2
1


Boxplot of color_intensity
color_intensity
12
10
8
6
4
2

[17]:
```
#6.For at least three selected features (e.g., alcohol, malic_acid,␣
↪color_intensity), Determine whether the distributions are symmetric,␣
↪positively skewed, or negatively skewed.
df[features].skew()
#Interpretation:
    #1.alcohol → slightly symmetric
    #2.malic_acid → highly positively skewed
    #3.color_intensity → positively skewed
```

[17]:
```
alcohol           -0.051482
malic_acid         1.039651
color_intensity    0.868585
dtype: float64
```

[ ]:
```
#7.Which features require normalization or standardization before modeling? And␣
↪why?
df.iloc[:, :-1].describe()
#Answer:
#Most features need standardization
#Reason:
    #1.Different scales (e.g., proline vs ash)
    #2.Distance-based & PCA models are scale-sensitive
```

[ ]:
```
          alcohol  malic_acid         ash  alcalinity_of_ash   magnesium  \
count  178.000000  178.000000  178.000000         178.000000  178.000000
mean    13.000618    2.336348    2.366517          19.494944   99.741573
std      0.811827    1.117146    0.274344           3.339564   14.282484
min     11.030000    0.740000    1.360000          10.600000   70.000000
25%     12.362500    1.602500    2.210000          17.200000   88.000000
50%     13.050000    1.865000    2.360000          19.500000   98.000000
75%     13.677500    3.082500    2.557500          21.500000  107.000000
max     14.830000    5.800000    3.230000          30.000000  162.000000

       total_phenols  flavanoids  nonflavanoid_phenols  proanthocyanins  \
count     178.000000  178.000000            178.000000       178.000000
mean        2.295112    2.029270              0.361854         1.590899
std         0.625851    0.998859              0.124453         0.572359
min         0.980000    0.340000              0.130000         0.410000
25%         1.742500    1.205000              0.270000         1.250000
50%         2.355000    2.135000              0.340000         1.555000
75%         2.800000    2.875000              0.437500         1.950000
max         3.880000    5.080000              0.660000         3.580000

       color_intensity         hue  od280/od315_of_diluted_wines      proline
```

```
count       178.000000  178.000000                    178.000000   178.000000
mean          5.058090    0.957449                      2.611685   746.893258
std           2.318286    0.228572                      0.709990   314.907474
min           1.280000    0.480000                      1.270000   278.000000
25%           3.220000    0.782500                      1.937500   500.500000
50%           4.690000    0.965000                      2.780000   673.500000
75%           6.200000    1.120000                      3.170000   985.000000
max          13.000000    1.710000                      4.000000  1680.000000
```

[19]:
```
#8.Compute the mean and standard deviation of each feature grouped by wine␣
↪class.
grouped_stats = df.groupby('target').agg(['mean', 'std'])
grouped_stats
```

[19]:
```
          alcohol           malic_acid                 ash            \
             mean       std       mean       std      mean       std
target
0       13.744746  0.462125   2.010678  0.688549  2.455593  0.227166
1       12.278732  0.537964   1.932676  1.015569  2.244789  0.315467
2       13.153750  0.530241   3.333750  1.087906  2.437083  0.184690

       alcalinity_of_ash             magnesium             …  \
                    mean       std        mean        std  …
target                                                     …
0              17.037288  2.546322  106.338983  10.498949  …
1              20.238028  3.349770   94.549296  16.753497  …
2              21.416667  2.258161   99.312500  10.890473  …

       proanthocyanins           color_intensity                 hue  \
                  mean       std            mean       std      mean
target
0             1.899322  0.412109        5.528305  1.238573  1.062034
1             1.630282  0.602068        3.086620  0.924929  1.056282
2             1.153542  0.408836        7.396250  2.310942  0.682708

                 od280/od315_of_diluted_wines                proline  \
             std                         mean       std         mean
target
0       0.116483                     3.157797  0.357077  1115.711864
1       0.202937                     2.785352  0.496573   519.507042
2       0.114441                     1.683542  0.272111   629.895833


               std
target
0       221.520767
1       157.211220
```

```
2       115.097043

[3 rows x 26 columns]
```

[20]:
```python
#9.Which features show the largest variation between classes?
between_class_variance = df.groupby('target').mean().var()
between_class_variance.sort_values(ascending=False)
#Answer:
    #1.proline
    #2.magnesium
    #3.alcalinity_of_ash
```

[20]:
```
proline                         100610.515200
magnesium                           35.176051
alcalinity_of_ash                    5.135480
color_intensity                      4.670660
flavanoids                           1.224197
malic_acid                           0.619935
od280/od315_of_diluted_wines         0.587688
alcohol                              0.544021
total_phenols                        0.337224
proanthocyanins                      0.142642
hue                                  0.047246
ash                                  0.013626
nonflavanoid_phenols                 0.006210
dtype: float64
```

[21]:
```python
#10.  Calculate the explained variance ratio
# Standardize data (exclude target)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df.iloc[:, :-1])
# Apply PCA
pca = PCA()
pca.fit(X_scaled)
# Create table
explained_variance_table = pd.DataFrame({
    'Principal Component': [f'PC{i+1}' for i in range(len(pca.
 ↪explained_variance_ratio_))],
    'Explained Variance Ratio': pca.explained_variance_ratio_,
    'Cumulative Explained Variance': np.cumsum(pca.explained_variance_ratio_)
})
explained_variance_table
```

[21]:
```
   Principal Component  Explained Variance Ratio  \
0                  PC1                  0.361988
1                  PC2                  0.192075
2                  PC3                  0.111236
```

```
3                   PC4                   0.070690
4                   PC5                   0.065633
5                   PC6                   0.049358
6                   PC7                   0.042387
7                   PC8                   0.026807
8                   PC9                   0.022222
9                  PC10                   0.019300
10                 PC11                   0.017368
11                 PC12                   0.012982
12                 PC13                   0.007952

    Cumulative Explained Variance
0                        0.361988
1                        0.554063
2                        0.665300
3                        0.735990
4                        0.801623
5                        0.850981
6                        0.893368
7                        0.920175
8                        0.942397
9                        0.961697
10                       0.979066
11                       0.992048
12                       1.000000
```

[22]:

```python
#BREAST CANCER DATASET
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
# Load dataset
cancer = load_breast_cancer()

# Create DataFrame
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
df['target'] = cancer.target   # 0 = malignant, 1 = benign

df.head()
```

[22]:

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030
```

```
   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  …  worst texture  worst perimeter  worst area  \
0                 0.07871  …          17.33           184.60      2019.0
1                 0.05667  …          23.41           158.80      1956.0
2                 0.05999  …          25.53           152.50      1709.0
3                 0.09744  …          26.50            98.87       567.7
4                 0.05883  …          16.67           152.20      1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   worst symmetry  worst fractal dimension  target
0          0.4601                  0.11890       0
1          0.2750                  0.08902       0
2          0.3613                  0.08758       0
3          0.6638                  0.17300       0
4          0.2364                  0.07678       0

[5 rows x 31 columns]
```

[23]:
```python
#1.What is the class distribution? Is the dataset balanced?
counts = df['target'].value_counts()
proportions = df['target'].value_counts(normalize=True)
result = pd.DataFrame({
    'Count': counts,
    'Proportion': proportions
})
result
#Answer:
    #1.Benign (1) > Malignant (0)
    #2.Dataset is moderately imbalanced, but acceptable
```

[23]:
```
        Count  Proportion
target
1         357    0.627417
0         212    0.372583
```

[24]:
```python
#2.Compute correlation between features and target variable.
corr_target = df.corr()['target'].sort_values()
corr_target
```

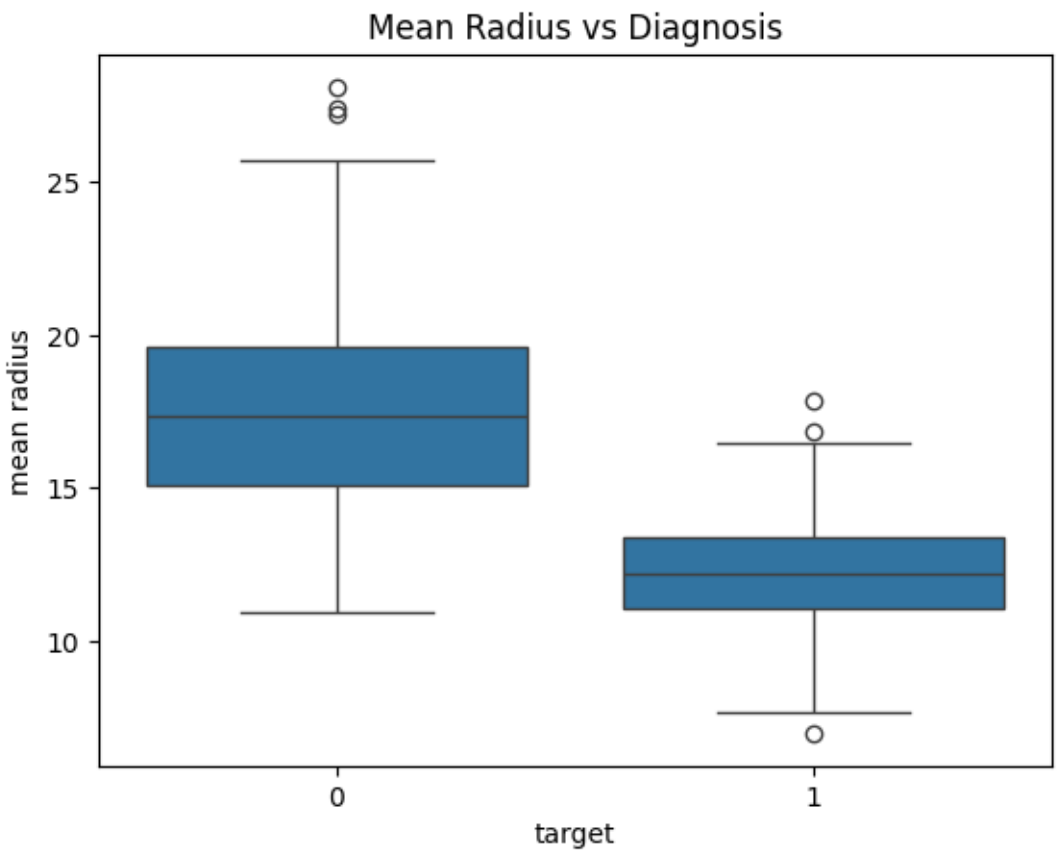
[24]:
```
worst concave points      -0.793566
worst perimeter           -0.782914
mean concave points       -0.776614
worst radius              -0.776454
mean perimeter            -0.742636
worst area                -0.733825
mean radius               -0.730029
mean area                 -0.708984
mean concavity            -0.696360
worst concavity           -0.659610
mean compactness          -0.596534
worst compactness         -0.590998
radius error              -0.567134
perimeter error           -0.556141
area error                -0.548236
worst texture             -0.456903
worst smoothness          -0.421465
worst symmetry            -0.416294
mean texture              -0.415185
concave points error      -0.408042
mean smoothness           -0.358560
mean symmetry             -0.330499
worst fractal dimension   -0.323872
compactness error         -0.292999
concavity error           -0.253730
fractal dimension error   -0.077972
symmetry error             0.006522
texture error              0.008303
mean fractal dimension     0.012838
smoothness error           0.067016
target                     1.000000
Name: target, dtype: float64
```

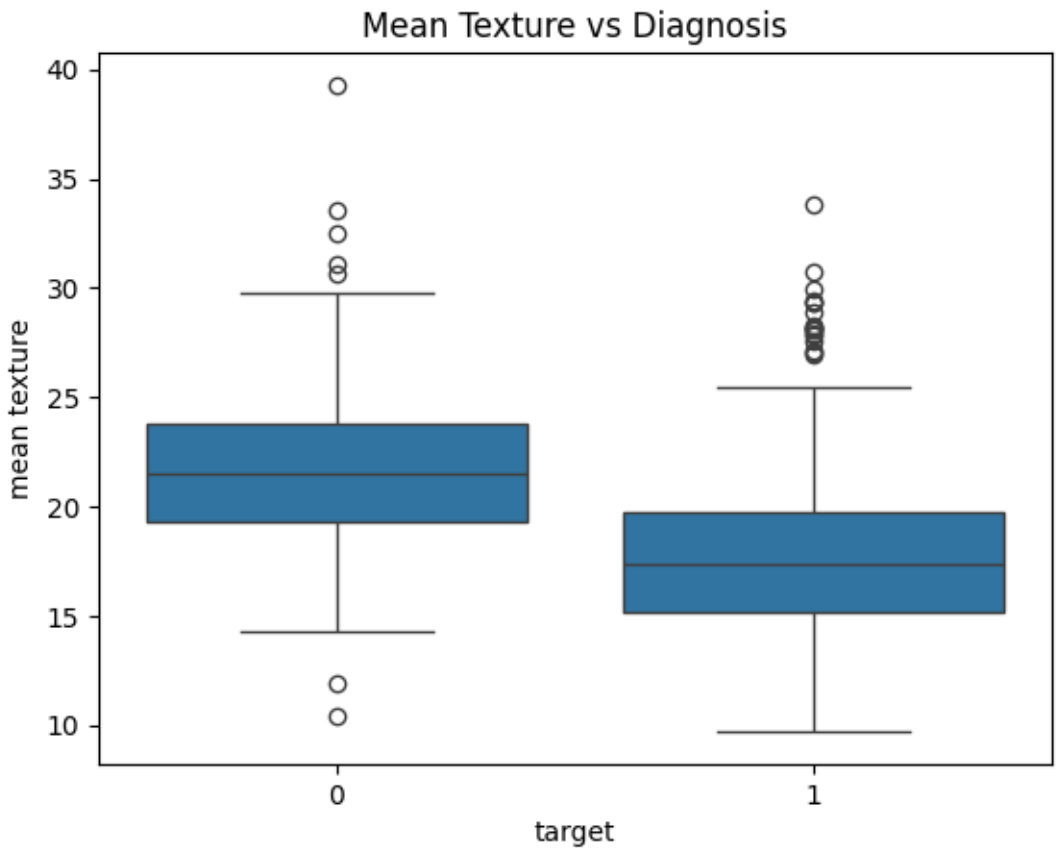
[25]:
```python
#3.Identify top 5 features most correlated with diagnosis.
# Absolute correlation
corr_target.abs().sort_values(ascending=False).head(6)
```

[25]:
```
target                  1.000000
worst concave points    0.793566
worst perimeter         0.782914
mean concave points     0.776614
worst radius            0.776454
mean perimeter          0.742636
```

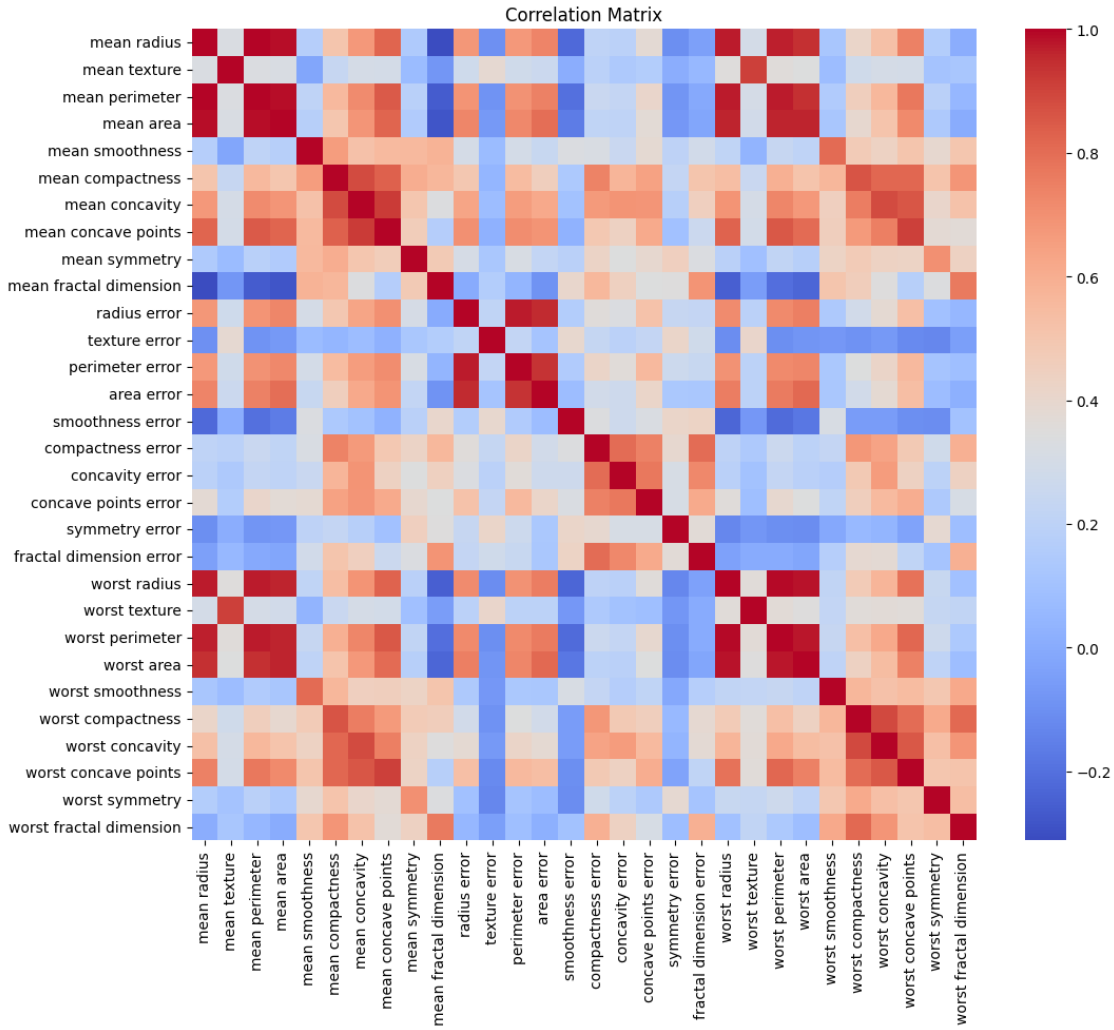
```
Name: target, dtype: float64
```

[ ]:
```
#4.Plot distribution of mean radius and mean texture grouped by class.
sns.boxplot(x='target', y='mean radius', data=df)
plt.title("Mean Radius vs Diagnosis")
plt.show()
sns.boxplot(x='target', y='mean texture', data=df)
plt.title("Mean Texture vs Diagnosis")
plt.show()
```

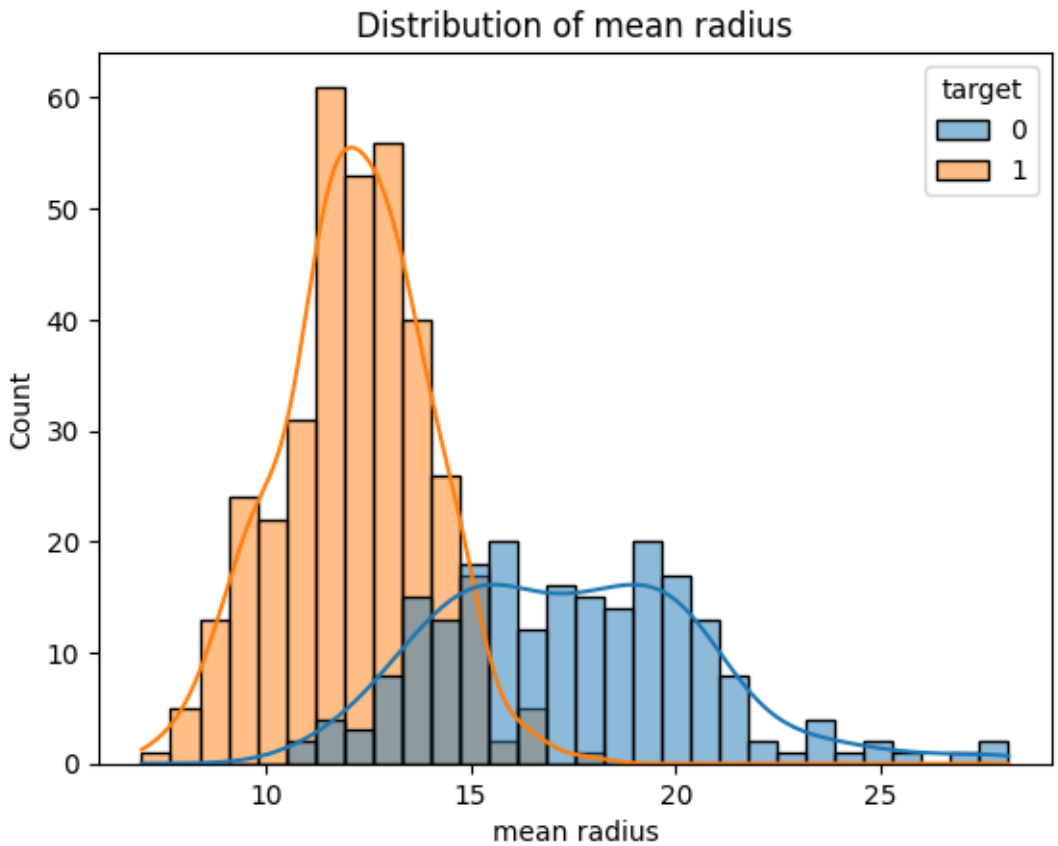


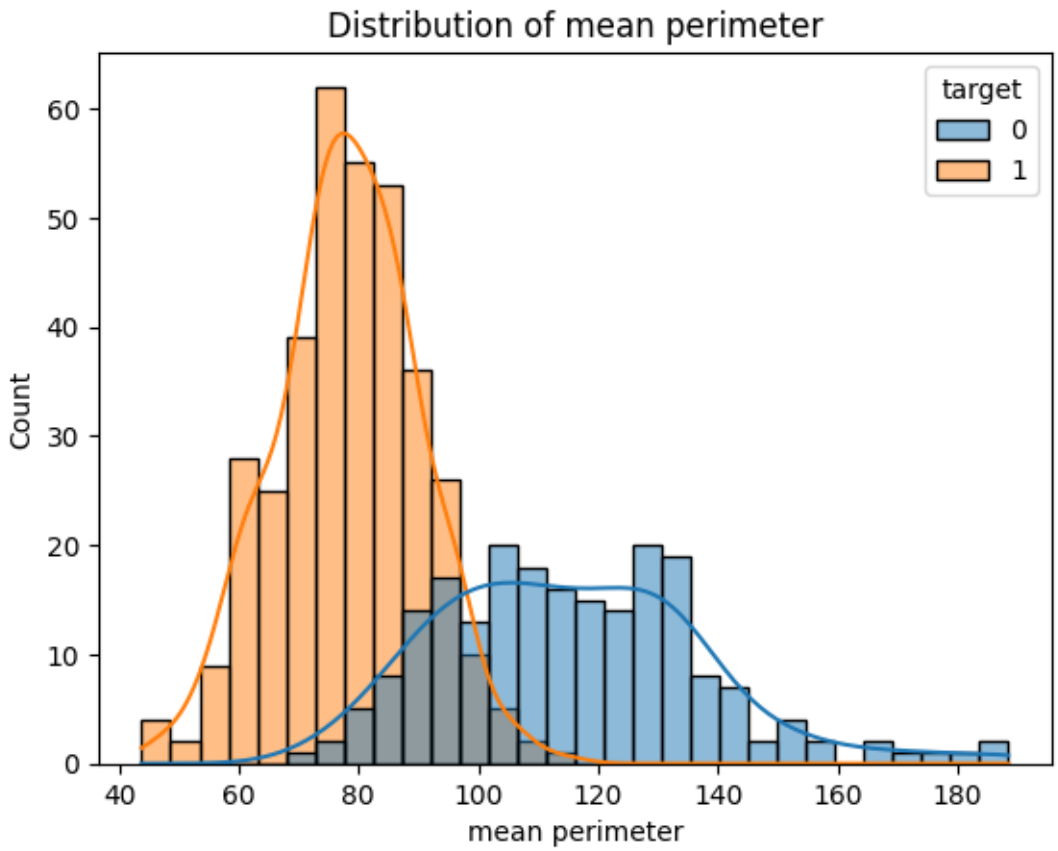


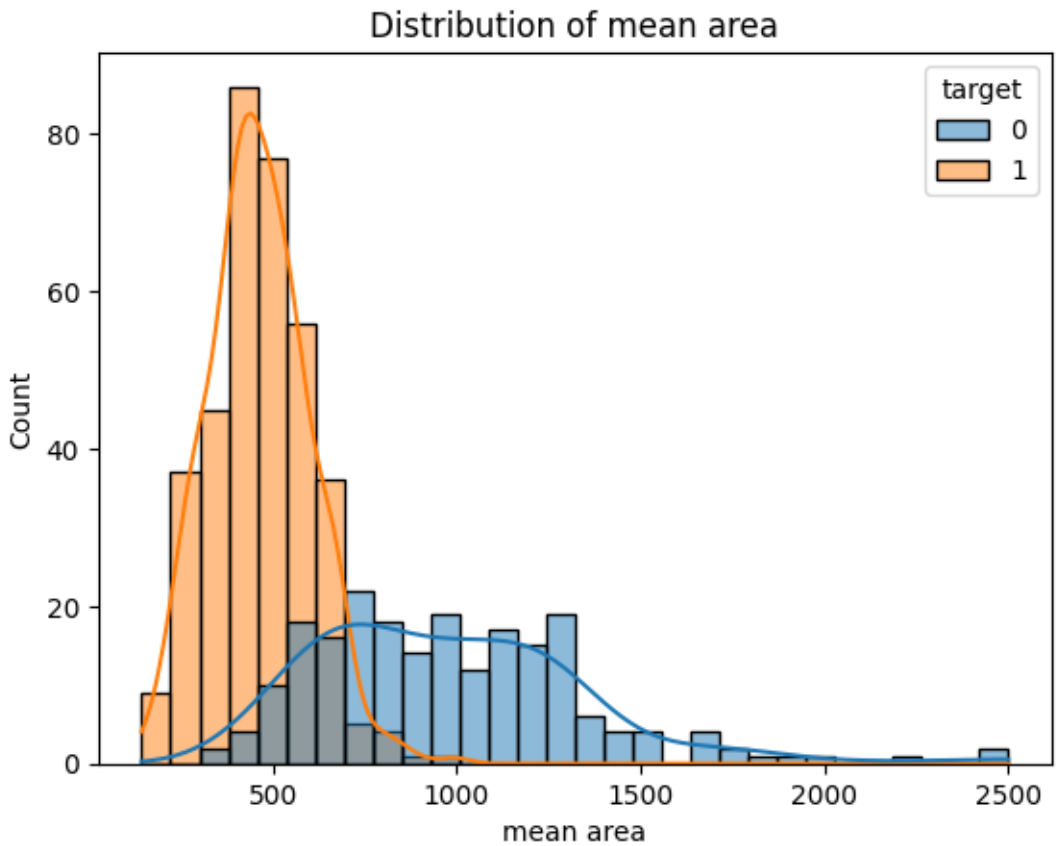
[27]:
```
#5.Detect multicollinearity using correlation matrix. Are there redundant␣
↪features
plt.figure(figsize=(12,10))
sns.heatmap(df.iloc[:, :-1].corr(), cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
#Observation:
    #1.Strong multicollinearity among:
        #mean radius, mean perimeter, mean area
        #Similar patterns for worst features
    #2.Redundant features exist
```



[28]:
```
#6.Plot histograms of three "mean" features (e.g., mean radius, mean perimeter,␣
↪mean area) grouped by diagnosis.
features = ['mean radius', 'mean perimeter', 'mean area']
for f in features:
    sns.histplot(data=df, x=f, hue='target', bins=30, kde=True)
    plt.title(f"Distribution of {f}")
    plt.show()
```


Distribution of mean radius
target
0
1
Count
mean radius
0
10
20
30
40
50
60
10
15
20
25


Distribution of mean perimeter
target
0
1
Count
0
10
20
30
40
50
60
40
60
80
100
120
140
160
180
mean perimeter



[29]:
```
#7.What differences do you observe between malignant and benign tumors?
#Observed:
    #1.Malignant tumors:
        #Larger radius, area, perimeter
        #Wider spread and right-skewed distributions
    #2.Benign tumors:
        #Smaller, more compact distributions
```

[30]:
```
#8.Calculate skewness for all features. Identify highly skewed features. Can␣
↪any transformation help these features.
# Select only feature columns (exclude target)
features = df.columns.drop('target')
# Skewness BEFORE transformation
skew_before = df[features].skew()
# Apply log1p transformation to all features
df_log = np.log1p(df[features])
# Skewness AFTER transformation
skew_after = df_log.skew()
# Combine results
```

```
skewness_comparison = pd.DataFrame({
    'Skewness Before': skew_before,
    'Skewness After Log': skew_after
}).sort_values(by='Skewness Before', ascending=False)
skewness_comparison
#Highly skewed features:
    #1.area error
    #2.worst area
    #3.mean area
#Helpful transformations:
    #1.Log transformation
    #2.Box-Cox transformation
```

[30]:

	Skewness Before	Skewness After Log
area error	5.447186	0.847298
concavity error	5.110463	4.231738
fractal dimension error	3.923969	3.874373
perimeter error	3.443615	1.025375
radius error	3.088612	1.713417
smoothness error	2.314450	2.283394
symmetry error	2.195133	2.135874
compactness error	1.902221	1.816278
worst area	1.859373	0.457614
worst fractal dimension	1.662579	1.558559
texture error	1.646444	0.629526
mean area	1.645732	0.287328
worst compactness	1.473555	1.048729
concave points error	1.444678	1.399869
worst symmetry	1.433928	1.152984
mean concavity	1.401180	1.206982
mean fractal dimension	1.304489	1.272294
mean compactness	1.190123	1.043195
mean concave points	1.171180	1.083180
worst concavity	1.150237	0.717939
worst perimeter	1.128164	0.462758
worst radius	1.103115	0.506347
mean perimeter	0.990650	0.328800
mean radius	0.942380	0.348744
mean symmetry	0.725609	0.632718
mean texture	0.650450	0.018733
worst texture	0.498321	-0.101178
worst concave points	0.492616	0.386721
mean smoothness	0.456324	0.406548
worst smoothness	0.415426	0.346313

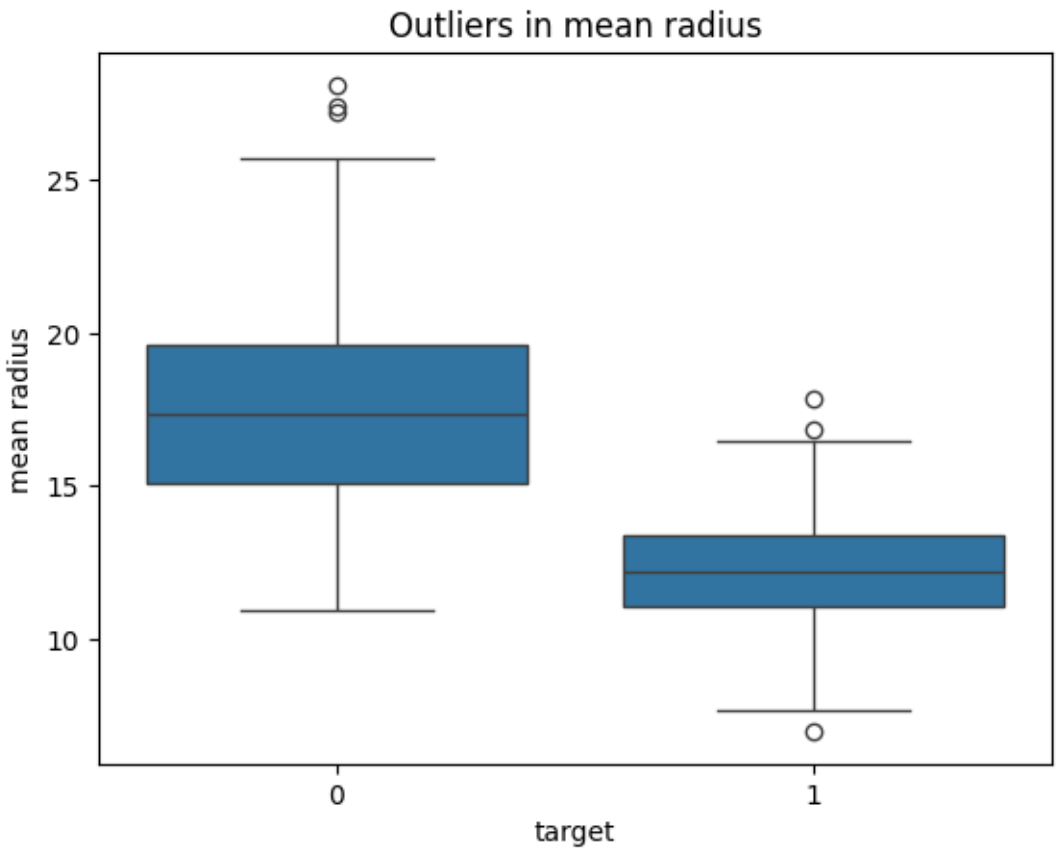
[31]:

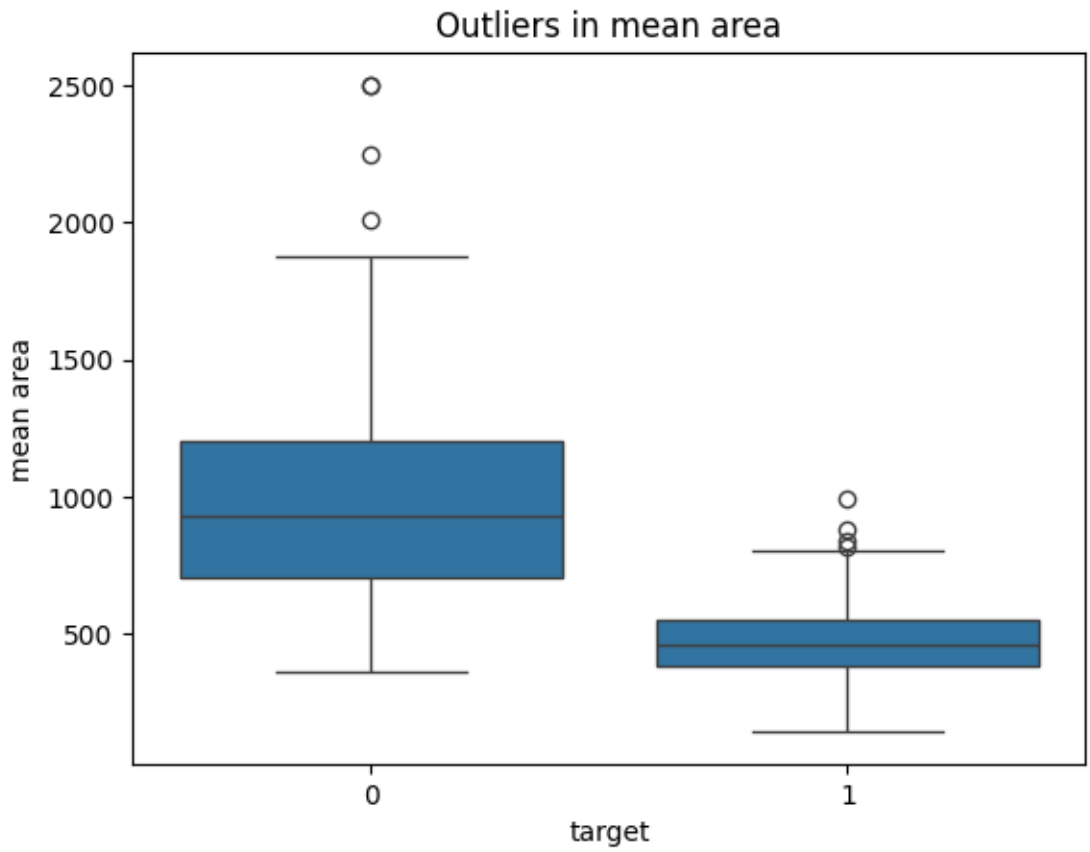
```
#9.Encode the target (malignant=1, benign=0) and compute correlation␣
↪coefficients. Which 5 features show strongest positive/negative correlation?
```

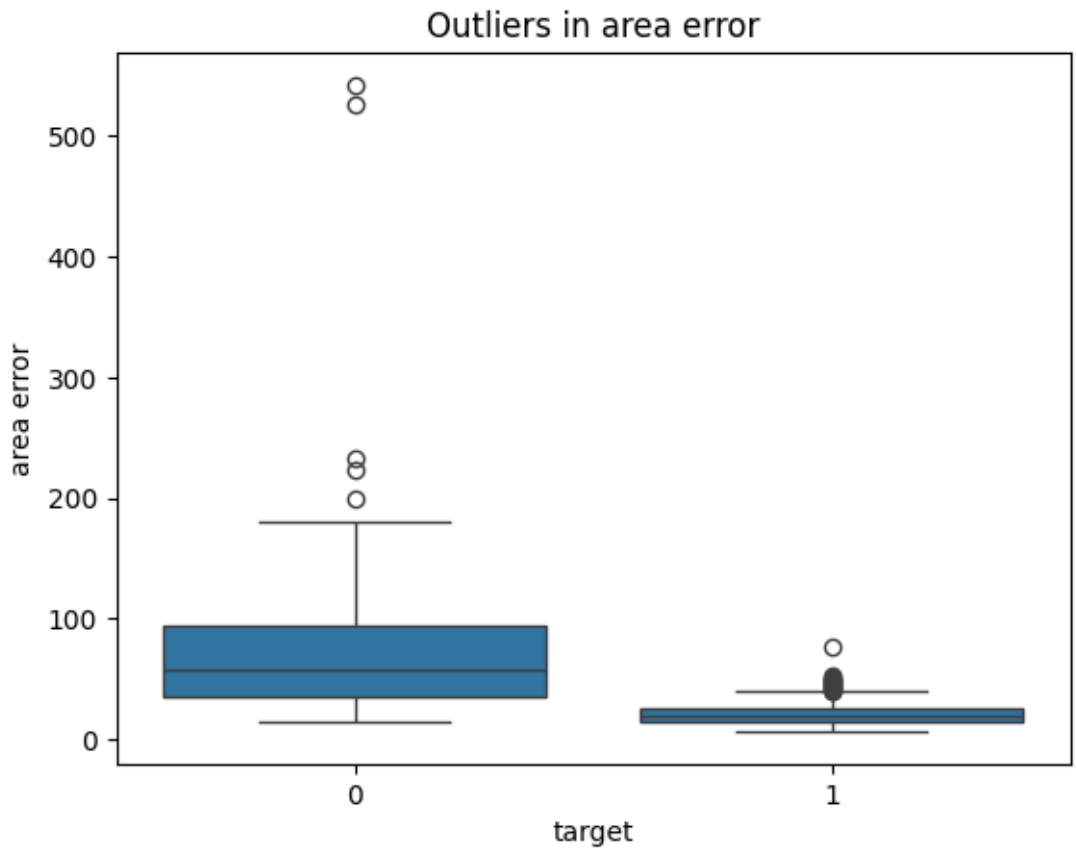
```python
# Encode target: malignant = 1, benign = 0
df['target_encoded'] = df['target'].map({0: 1, 1: 0})

# Correlation with encoded target
corr = df.corr()['target_encoded'].drop('target_encoded')

# Top 5 positive correlations (malignancy)
positive_corr = corr[corr > 0].sort_values(ascending=False).head(5)

# Top 5 negative correlations (benign)
negative_corr = corr[corr < 0].sort_values().head(5)

print("Top 5 POSITIVE correlations (associated with malignant tumors):\n")
print(positive_corr)

print("\nTop 5 NEGATIVE correlations (associated with benign tumors):\n")
print(negative_corr)
```

```
Top 5 POSITIVE correlations (associated with malignant tumors):

worst concave points    0.793566
worst perimeter         0.782914
mean concave points     0.776614
worst radius            0.776454
mean perimeter          0.742636
Name: target_encoded, dtype: float64

Top 5 NEGATIVE correlations (associated with benign tumors):

target                  -1.000000
smoothness error        -0.067016
mean fractal dimension  -0.012838
texture error           -0.008303
symmetry error          -0.006522
Name: target_encoded, dtype: float64
```

[32]:
```python
#10.Use boxplots or IQR method to detect outliers in at least three features.␣
↪Are outliers more common in malignant or benign class?
features = ['mean radius', 'mean area', 'area error']
for f in features:
    sns.boxplot(x='target', y=f, data=df)
    plt.title(f"Outliers in {f}")
    plt.show()
#Answer:
    #1.Outliers are more frequent in malignant class
    #2.Malignant tumors show higher variability
```


Outliers in mean radius
mean radius
25
20
15
10
0
1
target


Outliers in mean area
2500
2000
1500
1000
500
mean area
0
1
target


Outliers in area error
area error
500
400
300
200
100
0
0
1
target